Case Study: Modernising Global Technology Operations at BrightKids Toys

BrightKids Toys is a 35-year-old toy manufacturing company headquartered in Melbourne, with major operations in the United States, Germany, India, and Vietnam. It designs, manufactures, and distributes educational toys, electronic play kits, and seasonal products to retailers and direct customers in over 40 countries. Over the past five years, the company has expanded rapidly through e-commerce, partnerships with global marketplaces, and direct-to-school programs. Revenue has grown, but the technology landscape has become fragmented and difficult to manage.

The company's digital platform supports four critical business functions: global order management, warehouse inventory tracking, production planning, and customer support. These systems were built over time by different teams and vendors. Some are monolithic applications hosted on virtual machines, while others are newer services deployed in cloud environments. Because of acquisitions in Europe and North America, BrightKids now runs workloads in **two cloud providers (Oracle Cloud and AWS)** and one private data center.

The CIO has launched a "Global Platform **Reliability** and **Security** Program" to improve speed, resilience, and compliance. A cross-functional engineering team has been formed, and your student group has been hired as external consultants to propose a practical modernization approach. The board has **given six months for measurable progress** before next year's holiday season.

Current Technology State

The order management system is the most business-critical application. It receives traffic spikes during product launches and holiday sales. The current setup includes:

1. An order capture application.
2. An inventory and shipping estimate application.
3. A legacy monolith application for pricing and promotions.
4. A nightly ETL process that syncs data from regional warehouses.

These components communicate through a mix of REST APIs, batch jobs, point-to-point integrations, and direct database calls. Deployment is mostly manual: engineers SSH into servers, stop services, pull code, and restart processes. Configuration differs by region, leading to frequent "works in one country, fails in another" issues.

During the last holiday campaign, the company suffered three major incidents:

1. US orders were delayed for 16 hours due to a failed inventory sync.

2. European customers saw incorrect shipping prices caused by region-specific config drift.
3. A security alert identified exposed admin endpoints in a staging environment connected to production data.

The leadership team now wants standardized deployments, improved observability, stronger security controls, and clearer ownership boundaries between development, operations, and cloud providers.

Business Constraints

BrightKids cannot pause operations for a full rebuild. Any proposal must support gradual migration while keeping systems live. Additional constraints include:

1. Seasonal demand can increase traffic by 8–10x within days.
2. Data residency laws require some customer data to remain in-region.
3. The company must meet retailer SLA commitments (99.9% uptime for order APIs).
4. Engineering headcount is limited; solutions must be maintainable by existing teams.
5. A recent internal audit flagged weak change-management controls and inconsistent patching practices.

The CFO has approved budget for cloud modernization tools, but only if the business case shows reduced incident costs and lower deployment risk.

Emerging Challenges to Solve

1. Containerisation and Runtime Consistency

The platform currently runs across VMs with inconsistent dependencies. Services break during patch cycles because **package versions drift** by region. Teams want to adopt Docker to create immutable, reproducible runtime images and **reduce "snowflake servers."**

However, BrightKids has no standard base image policy, vulnerability scanning workflow, or image lifecycle governance. Teams are asking:

- Should each service own its Dockerfile independently?
- How should secrets be injected securely at runtime?
- What is the process for patching base images globally during critical CVEs?

2. Container Orchestration for Scale and Reliability

A pilot Kubernetes cluster was created by one regional team, but without standardized naming, security policies, or cluster templates. Workloads run, but there are no proper readiness probes, resource limits, or autoscaling policies. Incident reviews show cascading failures from one overloaded service affecting others.

The operations team needs a design that supports:

- Multi-region deployment patterns.
- Blue/green or canary deployments for safer releases.
- Centralized monitoring and alerting across clusters.
- High availability during zone or node failures.

Leadership is concerned that “Kubernetes complexity” may exceed team capability unless governance is clear and automation is strong.

3. Unified Integration Platform Across Global Systems

One of the biggest hidden problems is integration sprawl. BrightKids has over 120 integrations across ERP, WMS, CRM, shipping vendors, payment providers, and regional compliance systems. Most were built as one-off connectors using custom scripts, cron jobs, and direct API calls. Failures are difficult to trace, retries are inconsistent, and ownership is unclear.

A recurring issue occurs when a product catalog update in one region does not sync cleanly to downstream systems, causing **pricing mismatches** and **stock inaccuracies across countries**.

The architecture board is considering a unified integration platform to centralize:

- **API mediation and transformation.**
- **Event-driven integration between systems.**
- **Standardized retries, dead-letter queues, and failure handling.**
- **End-to-end observability for data flow across regions.**
- **Governance for connector reuse, schema contracts, and change control.**

The challenge is deciding whether to adopt an integration-platform-as-a-service (iPaaS), build an internal event platform, or use a hybrid model. Any approach must support legacy systems while enabling modern, secure integrations at scale.

Incident Triggering Executive Attention

Two weeks ago, a penetration test found that an internal admin API used by warehouse operators could be reached from a misconfigured network path. Authentication checks were weak, and logs showed repeated failed login attempts from foreign IP addresses. No confirmed breach occurred, but the finding was rated high risk due to potential impact on inventory integrity and shipment scheduling.

In parallel, a third-party dependency used by multiple services was reported with a critical vulnerability. Patching took 11 days globally because teams lacked a common container image pipeline and had no clear ownership for emergency rollout approvals.

These events prompted the CEO to issue a directive: “Modernize with urgency, but without disrupting holiday operations.”

Infrastructure Metrics and Target

The following table gives you some baseline and target information and metrics about the scenario:

Metric	Current State	Target (SLA)
Deployment Frequency	Manual	Weekly/On-demand Blue/Green or Canary
Mean Time to Recovery (MTTR)	16 hours (last incident for order delay)	< 1 hour
Uptime (Order APIs)	~98.5%	99.9%
Patching Cycle	11 days (Global)	< 24 hours (Critical)

Questions for Students to Present

1. What target architecture would you propose for BrightKids over the next 6 months and 18 months, and why?
2. How should BrightKids govern APIs for consistency, security, and partner compatibility?
3. What unified integration platform strategy would you recommend (iPaaS, event platform, or hybrid), and how would you migrate from point-to-point integrations safely?
4. What KPIs would you track to prove modernization success to executives (reliability, security, deployment speed, integration success rate, and recovery readiness)?